

ELECTRONICS GUIDE | جزوهٔ آموزشی مهندسی الکترونیک

راهنمای جامع
شتاب‌دهی اجرا با ICache
در CH32H417

از کارکرد حافظهٔ نهان دستورالعمل در هستهٔ QingKe V5، تا کنترل آن با رجیستر راهبرد و کانال‌های PMP، پاک‌سازی پس از ریست و مقایسهٔ عملکرد CoreMark

در این جزوه، ترجمهٔ فارسی سند رسمی AN03000 (نسخهٔ ۱٫۰، منتشرهٔ شرکت WCH) را می‌خوانید: CH32H417 به‌طور پیش‌فرض برنامه را در RAM بارگذاری می‌کند؛ اما با منابع FLASH 960K و RAM 896K، فشار بر حافظهٔ RAM محسوس است. با کش‌کردن دستورالعمل‌ها از فلش می‌توان این فشار را کاهش داد و سرعت اجرا را نزدیک به ITCM رساند: سازوکار کش، کنترل با CACHE_STRTG_CTLR و PMP، الزام پاک‌سازی پس از ریست و نتایج CoreMark گام‌به‌گام بررسی می‌شود.

هستهٔ QingKe V5 · ICache 32K · CACHE_STRTG_CTLR · کانال‌های PMP · NAPOT / NA4 / PMP · CoreMark · ITCM · واژه‌نامهٔ فنی

ترجمهٔ فارسی AN03000 V1.0 · نسخهٔ ۱٫۰ — آبان ۱۴۰۵

فهرست مطالب — برای پرش به بخش موردنظر، روی ردیف آن کلیک کنید؛ شمارهٔ صفحه‌ها بر اساس صفحات فیزیکی PDF است.

فهرست مطالب

۱ توضیحات و دامنهٔ کاربرد ۳

۲ بخش ۱ | معرفی کارکرد کش در هستهٔ V5 ..... ۳

۳ بخش ۲ | روش استفاده از کش ۴

2.1 کنترل کش با راهبرد حافظهٔ نهان ۴

2.2 کنترل کش با کانال‌های PMP ۶

۴ بخش ۳ | مقایسهٔ عملکرد کش ۱۰

3.1 مقایسهٔ CoreMark با کش در هستهٔ V5 ..... ۱۰

3.2 مقایسهٔ عملکرد کش در آزمون خطی ۱۲

۵ تاریخچهٔ نسخه‌ها و بیانیه ۱۲

۶ واژگان کلیدی (واژه‌نامهٔ فنی) ۱۳

فهرست شکل‌ها، جدول‌ها و کدها

شکل ۲-۱ تعریف بیت‌های رجیستر راهبرد حافظهٔ نهان ۴

شکل ۲-۲ تعریف بیت‌های رجیستر عملیات حافظهٔ نهان ۵

شکل ۲-۳ حالت‌های PMP ۶

شکل ۲-۴ تعریف بیت‌های رجیستر آدرس PMP ۷

شکل ۲-۵ تعریف بیت‌های رجیستر پیکربندی PMP ۷

شکل ۲-۶ رجیستر پوشش راهبرد کشِ PMP ۸

شکل ۳-۱ نتیجهٔ CoreMark با اجرا از FLASH ۱۰

شکل ۳-۲ نتیجهٔ CoreMark با FLASH+Cache ۱۱

شکل ۳-۳ نتیجهٔ CoreMark در ITCM ۱۱

جدول ۱ مقایسهٔ عملکرد کش ۱۲

کد ۱ پاک‌سازی و فعال‌سازی کش ۵

کد ۲ کش‌کردن بازهٔ کد با PMP ۸

کد ۳ تعریف بخش .text1 در فایل LD ۹

چکیده — برای تراشهٔ CH32H417 معمولاً برنامه به‌طور پیش‌فرض در RAM بارگذاری می‌شود تا کارایی بالا برود؛ اما با نگاه به کل منابع — FLASH 960K و RAM (SRAM+ITCM+DTCM) برابر 896K — منابع RAM نسبتاً کمتر است و نمی‌توان همهٔ آن را برای بارگذاری و اجرای برنامهٔ فلش به‌کار گرفت. در این وضعیت می‌توان بخشی از کد را از طریق حافظهٔ نهانِ دستورالعمل (Cache) کش کرد و فشار بر منابع RAM را به‌اندازهٔ مناسبی کاهش داد.

مقدمه

توضیحات و دامنهٔ کاربرد

برای تراشهٔ CH32H417 معمولاً برنامه به‌طور پیش‌فرض در RAM بارگذاری می‌شود تا کارایی بالا برود؛ اما با نگاه به کل منابع — فلش 960K و حافظهٔ RAM (SRAM+ITCM+DTCM) برابر 896K — منابع RAM نسبتاً کمتر است و همهٔ آن هم نمی‌تواند برای بارگذاری و اجرای برنامهٔ فلش مصرف شود. در چنین شرایطی می‌توان بخشی از کد را با حافظهٔ نهانِ دستورالعمل کش کرد و فشار بر منابع RAM را به‌اندازهٔ مناسبی کاست.

جدول ۱ دامنهٔ کاربرد سند

دامنهٔ کاربرد	سری
میکروکنترلرهای همگانی‌منظوره	CH32H417

بخش ۱

معرفی کارکرد کش در هستهٔ V5

◆ ۱.۱ معرفی کارکرد کش

Icache حافظهٔ بسیار کوچک اما بسیار سریعی درون CPU است که اندازهٔ آن معمولاً ثابت است و در H417 برابر 32K می‌باشد.

پس از فعال‌شدن کش، جریان کار به این شکل است: نخست CPU درخواست واکشی آدرس می‌دهد؛ به‌جای دسترسی مستقیم به آدرس کند، ابتدا در Icache «جست‌وجو» و «مقایسه» انجام می‌شود. اگر برچسب آدرس درون Icache با برچسب آدرسِ هدفِ دسترسی یکسان باشد، یعنی برخورد با کش (hit) رخ داده است. اگر برچسب‌ها نابرابر باشند یعنی خطای کش (miss) و در این حالت خط لولهٔ CPU متوقف می‌شود، درخواست خواندن حافظه انجام می‌گیرد و بلوک پیوستهٔ خوانده‌شده در آدرس شاخص Icache ذخیره و مقدار برچسب نیز به‌روز می‌شود. دفعهٔ بعد که CPU به همان آدرس برسد، اگر با برچسب آدرسِ Icache بخواند، دیگر نیازی به خواندن اضافی حافظه نیست. چون اندازهٔ هر Icache line معمولاً ۶۴ بایت است، ۱۵ دستور بعدی نیز پیوسته برخورد خواهند داشت.

سرعت اجرای فلش در H417 نسبتاً پایین است؛ بنابراین هنگام اجرا از فلش، روشن‌کردن کارکرد Icache می‌تواند سرعت واکشی و اجرای CPU را به‌طور مؤثر بالا ببرد.

بخش ۲

روش استفاده از کش

◆ ۲.۱ کنترل کش با راهبرد حافظهٔ نهان

کنترل کش با راهبرد حافظهٔ نهان یعنی پیکربندی رجیستر CACHE_STRTG_CTLR در آدرس 0xBC2. با تنظیم بیت‌های متناظر این رجیستر، CPU برای نواحی مختلف دستورالعمل‌ها را کش می‌کند. مزیت آن سادگی است: در حالت ماشین با نوشتن یک رجیستر انجام می‌شود. اما دامنهٔ کنترل آدرس بزرگ است؛ اگر کد واقعی از 32K (اندازهٔ Icache) بزرگ‌تر شود، خطای کش مکرر رخ می‌دهد و تا حدی کارایی اجرا پایین می‌آید.

◆ ۲.۱.۱ رجیسترها و بیت‌های پیکربندی مرتبط

تعریف رجیستر CACHE_STRTG_CTLR به شکل زیر است.

4.2.3.9 缓存策略控制寄存器（CACHE_STRTG_CTLR）

CSR 地址：0xBC2

位	名称	访问	描述	复位值
[31:28]	Reserved	RO	保留。	0
27	ic_mem1_strtg	MRW	对 0x80000000-0x9fffffff 区域的指令缓存使能： 1：允许对本区域内指令进行缓存；	1

V1.6 57 WCH

CH32H417 系列应用手册 https://wch.cn

			0：禁止对本区域内指令进行缓存。	
26	ic_mem0_strtg	MRW	对 0x60000000-0x7fffffff 区域的指令缓存使能。	1
25	ic_sram_strtg	MRW	对 0x20000000-0x3fffffff 区域的指令缓存使能。	1
24	ic_code_strtg	MRW	对 0x00000000-0x1fffffff 区域的指令缓存使能。	1
[23:2]	Reserved	RO	保留。	0
1	ic_disable	MRW	指令缓存失能标志位，为 0 开启指令缓存功能。	1
0	Reserved	RO	保留。	1

شکل ۲-۱ تعریف بیت‌های رجیستر راهبرد حافظهٔ نهان

بیت‌های bit[24:27] رجیستر، بیت‌های کنترل کش دستورالعملِ هر ناحیه هستند که به‌طور پیش‌فرض روشن‌اند و بر اساس نیاز واقعی پیکربندی می‌شوند؛ بیت bit[1] بیت فعال‌سازی کش دستورالعمل است و به‌طور پیش‌فرض خاموش است؛ با صفر کردن آن، کارکرد کش دستورالعمل فعال می‌شود.

تنها روشن‌کردن کش کافی نیست: با هر روشن‌شدن دوبارهٔ برق، کش به‌طور طبیعی درست کار می‌کند چون مقدار کش پس از روشن‌شدن به حالت تصادفی برمی‌گردد؛ اما اگر ریست نرم‌افزاری یا سخت‌افزاری انجام شود، داده‌های درون کش باقی می‌مانند و در این حالت برچسب‌ها (Tag) و دادهٔ دستورالعمل‌ها متعلق به برنامهٔ قدیمی‌اند. پس از ریست، هنگام واکشی دستور ممکن است CPU به‌اشتباه این خط‌های قدیمی را «برخورد» بپندارد، اجرا به‌هم بریزد و استثنای دستور غیرمجاز یا «پرواز» برنامه رخ دهد. برای حل این مسئله، پیش از فعال‌سازی کش دستورالعمل می‌توان محتوای کش را دستی پاک کرد. رجیستر آن در آدرس 0xBD0 است و باید بیت bit[2] آن یک شود.

4.2.3.15 缓存操作寄存器（OPCACHE_CTLR）

CSR 地址：0xBD0

位	名称	访问	描述	复位值
[31:5]	Vaddr[27:0]	WO	操作地址或索引信息。	0
[4:3]	Reserved	RO	保留。	0
2	Idx_mode	WO	索引模式 1：以 vaddr 值为地址信息执行操作； 0：以 vaddr 值中的索引信息为首地址执行操作。	0
[1:0]	Opcode[1:0]	WO	00：Icache invalidate; 其他：无效。	0

شکل ۲-۲ تعریف بیت‌های رجیستر عملیات حافظهٔ نهان

◆ ۲.۱.۲ نمونهٔ پیکربندی برنامه

```
/* Flush Cache */
li t0, 0x4
csrw 0xbd0, t0

/* Enable ICache */
li t0, 0x0E000002
csrc 0xbc2, t0
```

کد ۱ پاک‌سازی کش و فعال‌سازی کش دستورالعمل

این کد را در فایل startup_ch32h417_v5f.S قرار دهید تا هنگام مقداردهی اولیهٔ سیستم، کارکرد کش پیکربندی شود. کارکرد آن نخست محتوای کش را پاک می‌کند، سپس فقط برای ناحیهٔ 0x00000000-0x1fffffff کش دستورالعمل را فعال می‌کند و خود کارکرد را نیز روشن می‌سازد. در عمل می‌توانید بسته به نیاز، بیت‌های متناظر 0xbc2 را تنظیم کنید و ناحیهٔ کش‌شدنی را به‌صورت انعطاف‌پذیر انتخاب نمایید.

◆ ۲.۲ کنترل کش با کانال‌های PMP

PMP یعنی واحد حفاظت حافظهٔ فیزیکی؛ در معماری RISC-V برای افزایش امنیت سیستم، مجموعه‌ای از محدودسازی‌های دسترسی آدرس فیزیکی تعریف شده است که می‌تواند برای حافظهٔ فیزیکی نواحی، ویژگی‌های خواندن، نوشتن و اجرا تنظیم کند و حفاظت با طول ناحیهٔ حداقل ۴ بایت را برآورده سازد. هستهٔ QingKeV5 در پیکربندی کش می‌تواند از قابلیت تقسیم نواحی حفاظتیِ حافظه در PMP بهره بگیرد و درون کانال‌های نواحیِ تقسیم‌شده، کش دستورالعمل را فعال کند؛ به این ترتیب کارکرد کش انعطاف‌پذیرتر قابل استفاده است.

حالت‌های حفاظت حافظه در PMP سه‌تاست: TOR، NA4 و NAPOT. NA4 حفاظت ثابت ۴ بایتی است؛ NAPOT حفاظت 2^(G+2) بایتی با G≥1 است؛ اندازهٔ ناحیهٔ حفاظت در حالت TOR محدودیتی ندارد و مرزهای کانال حفاظت کامل با رجیسترهای آدرس پیکربندی می‌شود، پس این شیوهٔ نسبتاً انعطاف‌پذیرتر است. تفاوت دقیق سه حالت در شکل زیر آمده است.

A 值	名称	描述
00b	OFF	没有区域要保护
01b	TOR	顶端对齐区域保护： region = BUS_ADDR >> 2; pmp0cfg 下，0≤ region <pmpaddr0; pmp1cfg 下，pmpaddr0≤ region <pmpaddr1; pmp2cfg 下，pmpaddr1≤ region <pmpaddr2; pmp3cfg 下，pmpaddr2≤ region <pmpaddr3。 $pmpaddr_{i-1}$ = A_ADDR >> 2; $pmpaddr_i$ = B_ADDR >> 2。
10b	NA4	固定 4 字节区域保护。 pmp0cfg～pmp3cfg 对应 pmpaddr0～pmpaddr3 作为起始地址。 $pmpaddr_i$ = A_ADDR >> 2。
11b	NAPOT	保护 $2^{(G+2)}$ 区域，G≥1，此时 A_ADDR 为 $2^{(G+2)}$ 对齐。 $pmpaddr_i$ = （（A_ADDR >> 2）| （$2^{(G-1)}$ -1）。

شکل ۲-۳ حالت‌های PMP

◆ ۲.۲.۱ رجیسترها و بیت‌های پیکربندی مرتبط

چهار رجیستر آدرس PMP وجود دارد؛ در حالت TOR، آدرس صفر همراه با چهار رجیستر آدرس PMP می‌تواند چهار ناحیه تقسیم کند. رجیسترهای آدرس به شکل زیرند؛ توجه کنید که در عمل باید آدرس را ۲ بیت به راست شیفت داده و سپس بنویسید.

位	名称	访问	描述	复位值
[31:0]	ADDR0[31:0]	MRW	PMP 设置地址 0 的 bit[33:2]，实际高 2 位未用。	0

شکل ۲-۴ تعریف بیت‌های رجیستر آدرس PMP

رجیسترهای پیکربندی PMP نیز چهار مجموعه‌اند و هر یک به پیکربندی ناحیهٔ کانال متناظر خود مربوط است؛ به‌عنوان نمونه pmp0cfg[7:0] در شکل زیر آمده است. باید بیت ۵ (کارکرد کش دستورالعمل) پیکربندی شود، حالت PMP یعنی مقدار A در بیت‌های bit[4:3] انتخاب گردد (مطابق حالت‌های PMP در شکل ۲-۳) و بسته به نیاز، مجوزهای خواندن/نوشتن/اجرا تنظیم شود.

[7:0]	pmp0cfg[7:0]	MRW	位 / 名称 / 描述： 7 / L / 锁定使能，机器模式下可解锁 0：不锁定； 1：锁定相关寄存器。 6 / - / 保留。 5 / IC_Str / 地址匹配时执行指令缓存策略，策略值与 CACHE_PMP_OVR 寄存器对应。 [4:3] / A / 地址对齐及保护区域范围选择。 2 / X / 可执行属性。 1 / W / 可写入属性。 0 / R / 可读出属性。	0

شکل ۲-۵ تعریف بیت‌های رجیستر پیکربندی PMP

علاوه بر این‌ها باید رجیستر پوشش راهبرد کشِ PMP یعنی CACHE_PMP_OVR نیز پیکربندی شود؛ تعریف دقیق بیت‌ها در شکل زیر آمده است. پس از یک‌کردن جایگاه کانال متناظر، وقتی آدرس دستور تحت حفاظت کانال متناظر PMP باشد، دستور جاری قابل کش‌شدن است؛ توجه کنید پس از اتمام پیکربندی هنوز باید کارکرد کش دستورالعمل را روشن کنید — نوشتن صفر در بیت bit[1] رجیستر CACHE_STRTG_CTLR آن را فعال می‌کند.

CSR 地址：0xBC3

位	名称	访问	描述	复位值
[31:13]	Reserved	MRO	保留。	0
12	ic_pmp3cache_strtg	MRW	PMP 通道 3 地址区域的指令缓存策略寄存器。	0
[11:9]	Reserved	MRO	保留。	0
8	ic_pmp2cache_strtg	MRW	PMP 通道 2 地址区域的指令缓存策略寄存器。	0
[7:5]	Reserved	MRO	保留。	0
4	ic_pmp1cache_strtg	MRW	PMP 通道 1 地址区域的指令缓存策略寄存器。	0
[3:1]	Reserved	MRO	保留。	0
0	ic_pmp0cache_strtg	MRW	PMP 通道 0 地址区域的指令缓存策略寄存器： 1：当指令地址受 PMP 通道 0 保护时，当前指令可缓存； 0：当指令地址受 PMP 通道 0 保护时，当前指令不可缓存。	0

شکل ۲-۶ رجیستر پوشش راهبرد کشِ PMP

◆ ۲.۲.۲ نمونهٔ پیکربندی برنامه

```
/*Enable the cache to cache the code from _cache_beg to _cache_end */
/*PMP*/
la t0, _cache_beg
srli t0, t0, 2
csrw pmpaddr0, t0

la t0, _cache_end
srli t0, t0, 2
csrw pmpaddr1, t0

/*PMP channel 1*/
li t0, 0x10
csrw 0xbc3, t0

li t0, 0xAD00 /*lock*/
csrw 0x3a0, t0

/* Flush Cache */
li t0, 0x4
csrw 0xbd0, t0

/* Enable ICache */
li t0, 0x0F000002
csrc 0xbc2, t0
```

کد ۲ کش‌کردن کد بازهٔ _cache_beg تا _cache_end با کانال PMP

◆ ۲.۲.۳ خلاصهٔ کاربرد راهبرد کشِ کانال PMP

در نمونهٔ بخش ۲.۲.۲، مقدار برچسب‌های آدرس _cache_beg و _cache_end از طریق فایل LD تعریف و گرفته می‌شود؛ این‌ها آدرس ابتدا و انتهای ناحیهٔ مصرف‌شده از بخش فلش‌های تخصیص‌یافته‌اند. مثلاً در LD ناحیهٔ FLASH1 با آدرس شروع 0x08070000 و اندازهٔ 32K تقسیم می‌کنیم:

```
.text1 :
{
. = ALIGN(4);
KEEP(*(SORT_NONE(.text1)))
PROVIDE( _cache_beg = .);
*(.cache);
*(.cache.*);
PROVIDE( _cache_end = .);
. = ALIGN(4);
} >FLASH1 AT>FLASH1
```

کد ۳ تعریف بخش .text1 و برچسب‌های کش در فایل LD

پیش از تابع باید __attribute((section(".cache"))) اضافه شود تا پس از کامپایل در FLASH1 قرار گیرد. اگر حجم واقعی کد از 32K بیشتر شود، کامپایل خطا می‌دهد؛ کافی است اندازهٔ تخصیص FLASH1 را تنظیم کنید. توجه شود که در این حالت اندازهٔ ناحیهٔ کش دستورالعمل از ظرفیت کل کش فراتر رفته است، پس کارایی کل کش افت می‌کند.

برای نمونهٔ بخش ۲.۲.۲ می‌توان برچسب‌های آدرس _cache_beg و _cache_end را با مقادیر واقعی آدرس فلش جایگزین کرد. مثلاً در LD ناحیهٔ FLASH1 با آدرس شروع 0x08070000 و اندازهٔ 32K تقسیم شده باشد، کد چنین تغییر می‌کند:

```
la t0, 0x00070000
srli t0, t0, 2
csrw pmpaddr0, t0

la t0, 0x00078000
srli t0, t0, 2
csrw pmpaddr1, t0
```

در LD یک توصیف بخش جدید اضافه می‌کنیم تا توابع در این ناحیهٔ فلش قرار گیرند؛ پیش از تابع باید __attribute((section(".cache1"))) نوشته شود:

```
.text1 :
{
. = ALIGN(4);
KEEP(*(SORT_NONE(.text1)))
*(.cache1);
*(.cache1.*);
. = ALIGN(4);
} >FLASH1 AT>FLASH1
```

این شیوه به‌صورت پیش‌گیرانه 32K فلش برای بارگذاری دستورالعمل‌ها تخصیص می‌دهد؛ اگر حجم واقعی کد از 32K فراتر رود کامپایل خطا می‌دهد و در این حالت می‌توانید اندازهٔ تخصیصی FLASH1 و مقدار رجیستر pmpaddr1 را تنظیم کنید.

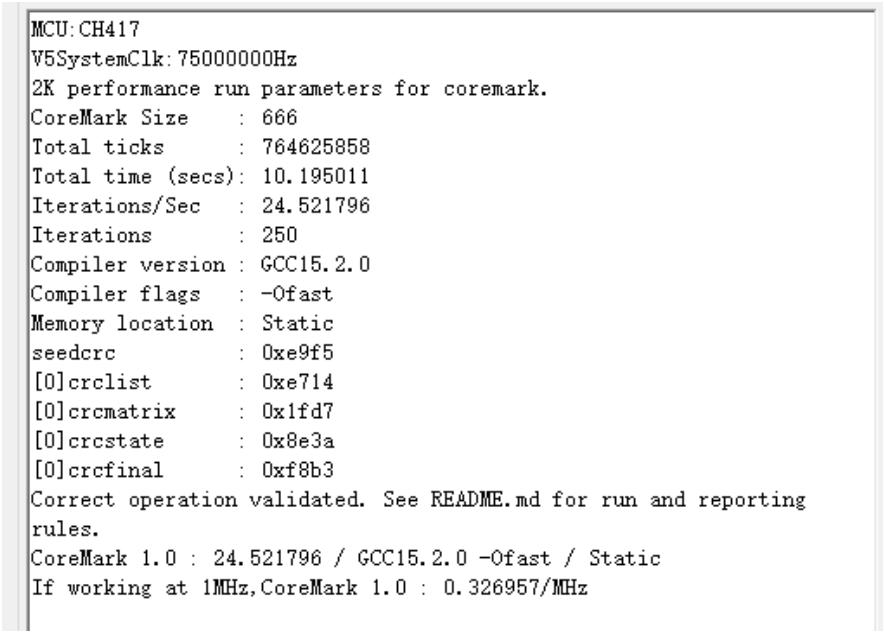
بخش ۳

مقایسهٔ عملکرد کش

۳.۱ مقایسهٔ CoreMark با کش در هستهٔ V5

دستورالعمل‌ها در فلش ذخیره شده‌اند و نتایج اجرای CoreMark مقایسه شده است؛ همه با استاندارد GCC15.

نتیجهٔ اجرای CoreMark با دستورالعمل‌های ذخیره‌شده در فلش: 0.327/MHz



شکل ۳-۱ نتیجهٔ CoreMark با اجرا از FLASH

نتیجهٔ اجرای CoreMark با دستورالعمل‌های ذخیره‌شده در فلش همراه با فعال‌سازی کش: 6.672/MHz

```
MCU:CH417
V5SystemClk:75000000Hz
2K performance run parameters for coremark.
CoreMark Size    : 666
Total ticks      : 824292378
Total time (secs): 10.990565
Iterations/Sec   : 500.429229
Iterations       : 5500
Compiler version : GCC15.2.0
Compiler flags   : -Ofast
Memory location  : Static
seedcrc          : 0xe9f5
[0]crclist       : 0xe714
[0]crcmatrix     : 0x1fd7
[0]crcstate      : 0x8e3a
[0]crcfinal      : 0x33ff
Correct operation validated. See README.md for run and reporting rules.
CoreMark 1.0 : 500.429229 / GCC15.2.0 -Ofast / Static
If working at 1MHz,CoreMark 1.0 : 6.672389/MHz
```

شکل ۳-۲ نتیجهٔ CoreMark با FLASH+Cache

نتیجهٔ اجرای CoreMark با دستورالعمل‌های ذخیره‌شده در ناحیهٔ نزدیک‌جوش ITCM: 6.802/MHz

```
MCU:CH417
V5SystemClk:400000000Hz
2K performance run parameters for coremark.
CoreMark Size    : 666
Total ticks      : 850476204
Total time (secs): 12.863608
Iterations/Sec   : 2720.853897
Iterations       : 35000
Compiler version : GCC15.2.0
Compiler flags   : -Ofast
Memory location  : Static
seedcrc          : 0xe9f5
[0]crclist       : 0xe714
[0]crcmatrix     : 0x1fd7
[0]crcstate      : 0x8e3a
[0]crcfinal      : 0x4983
Correct operation validated. See README.md for run and
reporting rules.
CoreMark 1.0 : 2720.853897 / GCC15.2.0 -Ofast / Static
If working at 1MHz,CoreMark 1.0 : 6.802134/MHz
```

شکل ۳-۳ نتیجهٔ CoreMark در ITCM

از مقایسهٔ نتایج، بهبود کارایی بارگذاری و اجرای دستورالعمل‌های کش‌شده از فلش با کش بسیار چشمگیر است و به ناحیهٔ نزدیک‌جوش ITCM نزدیک می‌شود. حجم کد آزمون از 32K تجاوز نکرده است؛ اگر حجم واقعی محتوای کش‌شده از 32K

بیشتر شود، کارایی کش افت می‌کند.

◆ ۳.۲ مقایسهٔ عملکرد کش در آزمون خطی

یک تابع 12K بایتی در فلش قرار گرفته است؛ محتوای تابع با گسترش nop و اجرای خطی بدون حلقه است.

جدول ۱ مقایسهٔ عملکرد کش (زمان به میکروثانیه)

تعداد اجرای تابع	اجرای از فلش، کش خاموش (us)	اجرای از فلش، کش روشن (us)
۱	253.35	1619.32
۲	509.26	1627.05
۳	763.28	1634.77
...	...	...
۱۰	2545.90	1688.81
...	...	...
۲۰	5093.08	1766.01

از داده‌های جدول می‌بینیم سرعت اجرای از فلش بسیار یکنواخت است؛ بارگذاری نخستین کش زمان طولانی‌تری می‌برد، اما سرعت اجرا ۳۰ برابرِ فلش است. اگر حجم واقعی کد از اندازهٔ کل کش تجاوز نکند یا کش مکرر بارگذاری نشود، هرچه تعداد اجرا بیشتر شود، کارایی کل کش بهتر می‌شود.

پیوست

تاریخچهٔ نسخه‌ها و بیانیه

جدول تاریخچه تاریخچهٔ نسخه‌های سند اصلی AN03000

تاریخ	نسخه	شرح تغییرات
2026/04/24	V1.0	انتشار نسخهٔ اول

◆ بیانیه

این راهنما صرفاً جهت مرجع ارائه شده است و نویسندهٔ سند اصلی می‌تواند بدون اطلاع قبلی و در هر زمان، تغییراتی در محتوای آن اعمال کند. ترجمهٔ فارسی این سند توسط تیم معمار الکترونیک صنعت تهیه شده است؛ برای اطمینان از به‌روزترین نسخه به https://wch.cn مراجعه کنید.

پیوست

واژگان کلیدی (واژه‌نامهٔ فنی)

اصطلاحات فنی سخت‌افزار مطابق سند اصلی به‌صورت انگلیسی حفظ شده‌اند. جدول زیر معادل‌ها و شرح کوتاه هر واژه را برای مرجع سریع جمع می‌کند.

جدول واژگان — واژگان کلیدی به‌کاررفته در این جزوه

واژه	یعنی چه؟
Icache	حافظهٔ نهان دستورالعمل؛ حافظهٔ کوچک ولی بسیار سریع درون CPU با ظرفیت ثابت 32K در H417
Cache line / Tag	بلوک ۶۴ بایتیِ بارگذاری‌شده در کش / برچسب آدرس برای تشخیص «برخورد» یا «خطا» در جست‌وجو
CACHE_STRTG_CTLR	رجیستر راهبرد کش در آدرس 0xBC2؛ بیت‌های [24:27] نواحی کش و بیت [1] فعال‌سازی کش
OPCACHE_CTLR (0xBD0)	رجیستر عملیات کش؛ با یک‌کردن بیت [2] پیش از فعال‌سازی، محتوای کش پاک می‌شود
PMP	واحد حفاظت حافظهٔ فیزیکی در معماری RISC-V؛ تعیین مجوز خواندن/نوشتن/اجرا با حداقل دقت ۴ بایت
TOR / NA4 / NAPOT	سه حالت تطبیق آدرس PMP: بازهٔ دو آدرس / ۴ بایت ثابت / توان‌دوم هم‌تراز با پارامتر G
pmpaddr / pmpcfg	رجیسترهای آدرس و پیکربندی کانال‌های PMP؛ آدرس‌ها باید ۲ بیت به راست شیفت شوند
CACHE_PMP_OVR	رجیستر پوشش راهبرد کشِ کانال‌های PMP در 0xBC3؛ تعیین می‌کند کدام کانال کش‌شدنی است
CoreMark	بنچمارک استاندارد EEMBC برای سنجش کارایی MCU؛ نتیجه بر مبنای تکرار بر ثانیه و بر مگاهرتز
ITCM	حافظهٔ نزدیک‌جوشِ دستورالعمل؛ اجرا با سرعت نزدیک به کش با دسترسی بدون انتظار
csrw / csrc	دستورهای نوشتن/پاک‌سازی بیت در رجیسترهای CSR هستهٔ RISC-V
LD file	اسکریپت پیونددهنده؛ تعیین محل قرارگیری بخش‌های کد در حافظه و تعریف برچسب‌های _cache_beg/_cache_end

معمار الکترونیک صنعت

MEMAR ELECTRONIC SANAT (MES)

در مورد مطالب این جزوه و ترجمه، بدیهی است که از نقطه‌نظرات و پیشنهادهای شما عزیزان در این راستا خوشحال می‌شویم. همچنین اشکالات، کم‌وکاستی‌ها و پرسش‌های خود را در این مورد با ما در میان بگذارید تا در نسخه‌های بعدی لحاظ شوند.

با تشکر، حسن معماری

ترجمهٔ فارسی سند AN03000 V1.0 · نسخهٔ ۱٫۰ — تاریخ ساخت: ۱۴۰۵/۰۸/۱۹

راه‌های ارتباط با ما

گیت‌هاب	https://github.com/MEMAR-ELECTRONIC-SANAT
کانال تلگرام	https://t.me/Memar_Electronic
پیج اینستاگرام	https://www.instagram.com/Memar.Electronic
ایمیل	Memari.h@gmail.com